

Evaluating the Complex Reasoning and Performance Capabilities of Large Language Models for Test Case Generation

A Learning-Based Software Testing Suite for Benchmarking Generative AI in Automated Test Case Generation

Vijay Bala Mahalingam
Computing Technologies
SRM Institute of Science and
Technology
Chennai, Tamil Nadu, India
vm7798@srmist.edu.in

Harish Parthasarathy*
Computing Technologies
SRM Institute of Science and
Technology
Chennai, Tamil Nadu, India
hp5491@srmist.edu.in

Aakash Baskaran
Computing Technologies
SRM Institute of Science and
Technology
Chennai, Tamil Nadu, India
ab7538@srmist.edu.in

Bharath Chandrasekaran
Computing Technologies
SRM Institute of Science and
Technology
Chennai, Tamil Nadu, India
bc9964@srmist.edu.in

Muthukrishnan Rajeshkumar
Computing Technologies
SRM Institute of Science and
Technology
Chennai, Tamil Nadu, India
mr7718@srmist.edu.in

Abstract

Modern large language models (LLMs) have demonstrated the ability to generate software test cases, but their reasoning complexity and performance vary widely. This paper presents a systematic evaluation of ten state-of-the-art generative AI models on the task of automated test case generation. We introduce a unified benchmarking framework that measures both the quality of the generated tests – via a complexity score and coverage of key testing operations – and the performance – via response time and output quantity. Our experiments reveal significant differences: some models produce highly complex, thorough test suites (with extensive assertions and even error-handling cases) but at slower speeds, while others respond faster yet with simpler, less rigorous tests. These findings underscore the importance of specialized evaluation methods for AI-driven software testing and demonstrate how our Learning-Based Software Testing (LBST) suite can benchmark LLMs' complex reasoning capabilities in generating test cases.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

Generative AI, Large Language Models (LLMs), Model evaluation, Automated test generation, AI-driven software testing, Test case

generation, Reasoning complexity, Complexity score, Operation coverage, Edge case handling

ACM Reference Format:

Vijay Bala Mahalingam, Harish Parthasarathy, Aakash Baskaran, Bharath Chandrasekaran, and Muthukrishnan Rajeshkumar. 2026. Evaluating the Complex Reasoning and Performance Capabilities of Large Language Models for Test Case Generation: A Learning-Based Software Testing Suite for Benchmarking Generative AI in Automated Test Case Generation. In *2026 10th International Conference on Control Engineering and Artificial Intelligence (CCEAI 2026)*, January 30–February 01, 2026, Hong Kong, Hong Kong. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3793928.3801117>

1 Introduction

The rise of generative AI models has opened new possibilities in software engineering, including automated test case generation. Traditionally, creating a robust suite of test cases for a piece of code is a labor-intensive process requiring deep understanding of the code's logic, edge cases, and potential failure modes. Large language models – trained on vast code corpora and natural language – can potentially learn these patterns and generate meaningful test cases automatically. However, evaluating whether these models truly exhibit complex reasoning (e.g., covering edge cases, using assertions appropriately) and how they perform under practical constraints (e.g., time taken to generate tests) remains a challenge.

This paper addresses the evaluation gap by assessing multiple modern LLMs on their ability to generate unit tests for given code. We focus on two aspects: (1) Reasoning complexity, i.e., how sophisticated and thorough the generated tests are, and (2) Performance, i.e., how efficient and productive the models are in doing so. Unlike general code generation or QA tasks, test generation requires the model not only to produce syntactically correct code, but to infer a variety of test scenarios (including normal and edge cases) and validate outcomes using assertions or error checks. These requirements make it an excellent proxy for a model's deeper reasoning about code behavior.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. CCEAI 2026, Hong Kong, Hong Kong
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2177-9/26/01
<https://doi.org/10.1145/3793928.3801117>

We evaluate ten cutting-edge models from a range of providers and research labs, including both proprietary and open-source systems: Claude Sonnet 4, GPT-5, LLaMA-4-Maverick, Phi 4, Qwen3-32B, DeepSeek-V3, Grok 3, Cohere Command R+, Gemini 2.5 Flash, and Mistral Medium 2505. Using our Learning-Based Software Testing (LBST) suite, we subject each model to identical test generation tasks and measure their outputs against a set of quantitative metrics. In summary, our contributions are:

- **Generalized Evaluation Methodology:** We develop a unified framework to benchmark LLMs on test case generation, defining metrics that capture both output quality and generation performance in a model-agnostic manner.
- **Multi-Model Comparative Study:** We apply this methodology to a diverse set of ten modern AI models, providing the first side-by-side comparison of their test generation capabilities.
- **Insights on Reasoning vs Performance:** Through detailed analysis, we highlight trade-offs (e.g., reasoning depth versus speed) and pinpoint specific strengths and weaknesses of each model in the context of software testing.
- **Guidance for AI-Assisted Testing:** Our findings inform practitioners and researchers about the current state-of-the-art for AI-generated tests, identifying which models are more suitable for reliable test generation and where improvements are needed.

The remainder of this paper is organized as follows. Section 2 discusses related work in automated test generation and AI-based software testing. Section 3 describes our evaluation methodology and metrics. Section 4 outlines the architecture of the LBST benchmarking suite. Section 5 details the experimental setup, including the models and test scenarios used. Section 6 presents the results with a comparative performance table. Section 7 provides an analysis and discussion of the findings. Section 8 acknowledges the limitations of our study, and Section 9 concludes the paper with final remarks and future directions.

2 Related Work

Our work sits at the intersection of automated test generation and LLM-based code intelligence. We review the most relevant strands of prior research below.

2.1 Traditional Automated Test Generation

Automated test generation has been studied extensively long before the emergence of LLMs. Search-based software testing (SBST) applies metaheuristic search algorithms—such as genetic algorithms—to evolve test inputs that maximize structural coverage criteria. EvoSuite [17] is a prominent example that generates whole test suites for Java classes by optimizing for branch coverage. Symbolic and concolic execution tools such as DART [16] and KLEE [18] systematically explore program paths by treating inputs as symbolic values and solving path constraints to derive concrete test inputs. Random testing approaches, including QuickCheck [19] for Haskell and Randoop [20] for Java, generate large volumes of random inputs and check against specifications or implicit oracles (e.g., absence of crashes). While these techniques can achieve high code coverage, they often require significant setup (e.g., instrumented builds,

formal specifications) and may struggle with complex program logic or environment-dependent behavior. Unlike these approaches, LLM-based test generation requires no program instrumentation and can reason about intended behavior from natural language descriptions and code context alone.

2.2 LLM-Based Test Generation

The application of large language models to software testing has grown rapidly. Early work with OpenAI’s Codex [26] demonstrated that language models trained on code corpora could produce syntactically valid unit tests when prompted with function signatures and docstrings. Subsequent studies explored GPT-3.5 and GPT-4 for test generation, finding that these models could cover a meaningful portion of branches in small to medium-sized functions. Tools such as GitHub Copilot [27] have brought LLM-assisted test writing into mainstream developer workflows, and recent multi-agent frameworks [14] propose having separate LLM agents for test planning, generation, and validation. Wang et al. [15] provide a comprehensive survey of the rapidly expanding landscape, categorizing LLM applications in testing by task type (unit testing, integration testing, fuzzing, mutation testing) and identifying key open challenges such as oracle quality and flaky test avoidance. Our work contributes to this area by providing a controlled, head-to-head comparison of ten diverse models under identical conditions, which is a gap that existing studies have not addressed.

2.3 Benchmarks and Evaluation for LLM-Generated Tests

Evaluating the quality of AI-generated tests is itself a developing research area. Traditional metrics such as code coverage (line, branch, mutation score) are well-established for assessing test suites, but they require executable tests and a working codebase. Wang et al. [13] introduced TESTEVAL, a benchmark specifically designed for evaluating LLM-generated unit tests, measuring aspects such as correctness, coverage, and readability. Software complexity metrics—including McCabe’s cyclomatic complexity [21], Halstead’s software science metrics [22], and SonarSource’s cognitive complexity [23]—have traditionally been applied to production code rather than test code, but they inspire our complexity scoring approach. Maintainability indices [24] and graph-based program representations [25] offer additional lenses through which generated code can be assessed. Our LBST suite builds on these ideas by defining a composite complexity score tailored to test cases, capturing aspects such as input diversity, assertion usage, and error-handling coverage. This provides a lightweight, execution-free metric that complements traditional coverage-based evaluation and is applicable across any model or framework.

3 Methodology

Our evaluation methodology is designed to measure both the quality of test cases generated by AI models and the performance characteristics of the generation process. Rather than focusing on any single codebase or bug-finding scenario, we use a generalized approach that could be applied to arbitrary functions or modules. The methodology consists of the following steps:

Benchmark Test Scenarios: We prepared a set of sample Python functions representing typical scenarios for test generation. These included a mix of computational logic and potential edge conditions (for example, functions operating on data structures, handling different input types, etc.). The chosen functions are of moderate complexity to require non-trivial tests (including edge cases and error conditions) without exceeding model context limits.

Prompting and Test Generation: Each model was given identical instructions to generate test cases for the target function. The prompt asked the model to produce a suite of unit tests (up to a maximum of 10 tests) that verify the correctness of the function’s behavior. To ensure fairness, prompts were standardized and models were queried in a similar manner (e.g., using temperature settings that prioritize deterministic, reproducible outputs over creativity).

Structured Output and Parsing: We requested models to output tests in a structured format (specifically, a JSON with fields like test description, input, expected output, and the test code). This facilitated automated parsing. After generation, the raw output from each model was parsed by our LBST suite to extract the individual test cases and details.

Metric Computation: For each model’s output on each scenario, we computed a set of metrics capturing different aspects of performance and reasoning:

- **Response Time:** The time taken by the model to generate the test suite, measured from prompt dispatch to completed response. This includes both model inference time and any network latency inherent to the API call.
- **Test Case Count:** The number of distinct test cases produced (upper limit of 10). Duplicate or near-identical tests (e.g., tests differing only in variable names) were counted only once.
- **Test Case Complexity Score:** A composite score on a 1–10 scale reflecting the thoroughness and structure of the generated tests. The score is computed by evaluating several sub-factors: diversity of input values (including normal, boundary, and edge-case inputs), completeness of test specifications (presence of descriptive names, clear expected outputs), structural quality (use of setup and teardown patterns, test isolation), and the presence and variety of assertions. Each sub-factor contributes to the overall score, which is then normalized to the 1–10 range. Higher scores indicate tests that demonstrate deeper reasoning about the function’s behavior and potential failure modes.
- **Operation Coverage:** We track two specific coverage indicators. *Assertions Coverage* measures the percentage of generated tests that contain at least one assertion statement (e.g., `assert`, `assertEqual`). *Error Handling Coverage* measures the percentage of tests that explicitly test error conditions, such as invalid inputs, boundary violations, or expected exceptions (e.g., using `assertRaises` or `pytest.raises`).

Aggregation: Since each model was run on multiple test scenarios, we aggregate the metrics to obtain an overall picture of performance per model. In our study, we used three different target functions; thus, for each model we averaged the response times and other metrics across these runs. We report both the mean values and note any significant variance across scenarios where applicable.

By applying this methodology, we ensure a fair and comprehensive comparison of the models. All models see the same tasks and are evaluated in the same way, which isolates differences in their capabilities. The approach is generalized: it can be extended to more models or more complex scenarios with minimal changes, thanks to the standardized prompting and automated analysis.

4 Architecture

The Learning-Based Software Testing (LBST) suite provides the infrastructure for this evaluation. The architecture of LBST is modular, enabling integration of multiple AI models and automated analysis of their outputs. Figure 1 illustrates the overall system design. At a high level, the system is composed of the following components:

Model Integration Layer: This layer abstracts the API or interface for each AI model provider. Whether a model is accessed via a cloud API (like OpenAI, Anthropic, Google, etc.) or run locally through an inference server, the integration layer presents a unified method to send a prompt and receive a completion. Each provider adapter handles authentication, request formatting, rate limiting, and response normalization. This design allows new models to be added with minimal code changes—only a new adapter conforming to the common interface is required.

Prompt & Scenario Manager: This component stores the test generation prompts and the library of test scenarios (the code snippets to generate tests for). It ensures that each model receives the exact same prompt for a given scenario, eliminating prompt variation as a confounding factor. The manager supports parameterized prompts, allowing systematic variation of instructions (e.g., adjusting the requested number of tests or specifying output format) for future ablation studies. Scenario metadata, including function signatures, docstrings, and expected behavior descriptions, is also managed here.

Output Parser and Analyzer: Once a model returns its generated test cases, this module parses the output. Because we ask for structured JSON-like output, the parser can extract each test’s details, including the test name, input values, expected outputs, and the test code itself. The parser includes error-recovery logic to handle minor formatting inconsistencies (e.g., missing trailing commas, extraneous markdown fences) that different models may produce. The analyzer then computes metrics such as response time, complexity score, and checks for the presence of assertions and error-handling constructs by performing lightweight static analysis on the generated test code.

Data Aggregator and Interface: The results for each model and scenario are collected, aggregated, and presented. We implemented a Streamlit-based web interface for interactive visualization, allowing users to filter by model, scenario, or metric and to drill down into individual test cases. For this paper, the aggregator produced consolidated tables and charts summarizing cross-model performance. All raw data is stored in structured JSON format, enabling reproducibility and further analysis.

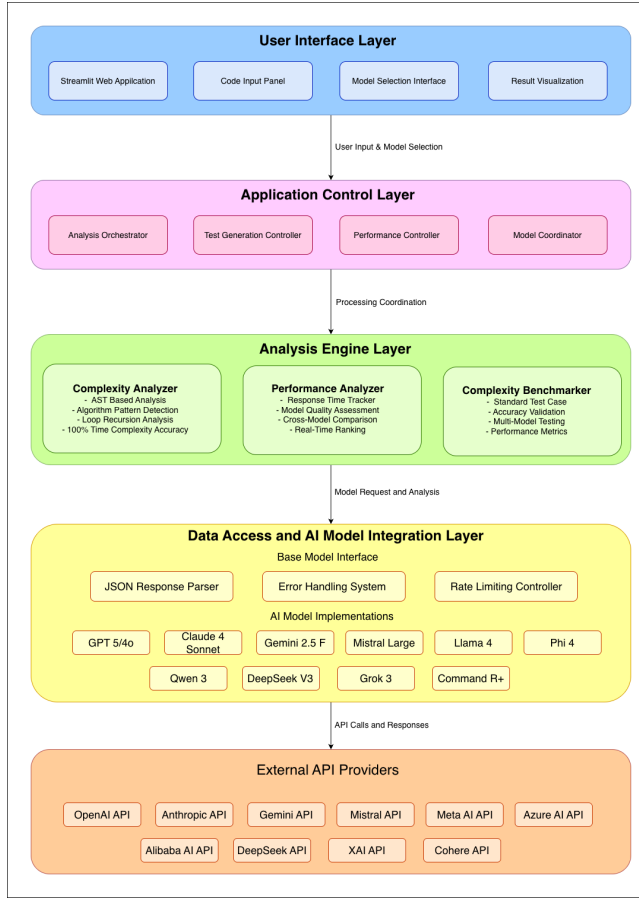


Figure 1: System Architecture of the LBST benchmarking suite, showing the Model Integration Layer, Prompt & Scenario Manager, Output Parser and Analyzer, and Data Aggregator components.

5 Experimental Setup

In this section, we describe the concrete setup for evaluating the ten AI models on test case generation. Our goal was to create a level playing field and meaningful challenge for the models:

Models Evaluated: We selected ten generative AI models that were among the most advanced or representative as of 2025. These include:

Claude Sonnet 4 – an advanced model from Anthropic, known for its long-form reasoning capabilities.

GPT-5 – OpenAI’s latest flagship model, with state-of-the-art performance on many language and coding tasks.

LLaMA-4-Maverick – an open-source model (Meta’s LLaMA 4 series, Maverick variant) around 17B parameters, fine-tuned for instruction following and coding.

Phi 4 – a model from Microsoft’s Azure AI (Phi series) oriented towards code understanding.

Qwen3-32B – Alibaba’s Qwen model (third generation), 32B parameter size, aimed at multilingual and code tasks.

DeepSeek-V3 – a specialized model (DeepSeek version 3) trained for code and problem solving.

Grok 3 – xAI’s Grok model (version 3), focusing on reasoning.

Cohere Command R+ – Cohere’s instruction-following LLM, research/enhanced variant.

Gemini 2.5 Flash – Google’s Gemini model (v2.5 Flash), efficient and strong at coding tasks.

Mistral Medium 2505 – Mistral AI’s medium-sized model geared towards coding assistance.

Test Functions: We selected three Python functions to serve as the subjects for test generation. The functions were chosen to represent a range of common programming patterns and testing challenges:

- A function involving data structure manipulation (e.g., sorting, filtering, or transforming lists and dictionaries), requiring the model to reason about different input shapes, empty collections, and ordering properties.
- A function containing potential error conditions (e.g., division, type conversion, or index-based access), where the model must identify inputs that could trigger exceptions and generate appropriate error-handling tests.
- A function with a performance or algorithmic aspect (e.g., recursive computation or search), where boundary conditions such as very large inputs, worst-case scenarios, and base cases are relevant.

Each function was of moderate complexity—sufficiently non-trivial to require thoughtful test design, yet short enough to fit comfortably within the context window of all evaluated models without truncation.

Prompt Format: A standardized prompt was crafted to instruct each model to generate up to 10 unit tests in a structured JSON format. The prompt specified that the tests should cover typical usage, edge cases, and error conditions, and that each test should include a descriptive name, input values, expected output, and the executable test code. The prompt did not provide example test cases, ensuring that the model had to rely entirely on its own reasoning to determine appropriate test scenarios. Temperature was set to 0 (or the closest deterministic setting available) for all models to minimize randomness and ensure reproducibility.

Measurement Procedure: Each model was run on each of the three functions sequentially, and response time was logged using wall-clock timing from prompt dispatch to response completion. To account for network variability, each model-function pair was queried three times and the median response time was used. Outputs were parsed and analyzed automatically by the LBST suite’s Output Parser and Analyzer module.

Validation of Outputs: Generated outputs were first checked by the automated parser for structural validity. Minor JSON formatting issues (such as missing commas, extraneous markdown code fences, or trailing text after the JSON block) were handled by the parser’s error-recovery logic. Outputs that could not be parsed even after recovery were flagged and manually inspected. In practice, all ten models produced parseable output for all three scenarios, though the degree of formatting compliance varied.

Table 1: Test Generation Performance of Evaluated Models

Model	Test Cases	Complexity Score	Response Time (s)	Coverage (%)
GPT-5 (w/ GPT-4 fallback)	10	5.8	50.5	67
Claude Sonnet 4	10	5.8	2.4	100
Llama-4-Maverick-17B	10	5.9	1.9	100
Phi 4	10	5.6	43.7	83
Qwen3-32B	10	5.7	8.6	100
DeepSeek-V3-0324	10	6.0	11.8	100
Grok 3	10	6.0	27.3	100
Cohere Command A	10	4.0	28.3	0
Gemini 2.5 Flash	10	5.5	9.4	50
Mistral Medium 2505	10	6.0	28.3	100

6 Results

Using the methodology and setup described, we collected extensive data on each model’s test generation performance. Table 1 summarizes results across four primary metrics: response time, number of test cases, complexity score, and operation coverage.

Table 1 shows comparison of models on test generation. “Test Cases” is the average number of tests generated (max 10). Complexity Score is on a 1–10 scale (higher is more complex/thorough). “Assertions Coverage” is the percentage of generated tests containing assertions; “Error Handling” is the percentage containing error/exception tests.

Several observations can be drawn from these results:

Response Time: There is a large variance in generation speed. The fastest model by far was LLaMA-4-Maverick, with an average of only ~1.9 seconds to produce a full test suite, followed closely by Claude Sonnet 4 (~2.4 s). These two were orders of magnitude faster than the slowest, GPT-5, which took about 50 seconds on average (likely due to its size and possibly network latency or fallback to a slower model). Other models showed moderate times: Qwen3 and Gemini were under 10 seconds, DeepSeek around 12 s, while Grok, Cohere, Mistral, and Phi ranged roughly 27–43 s.

Test Case Count: Nearly all models attempted to provide the maximum number of tests (10). Most achieved an average of 10.0 across scenarios. In general, the high counts show that the prompt for “up to 10 tests” was taken seriously by the models.

Complexity Score: The highest observed complexity score was 6.0 (on our 1–10 scale), achieved by DeepSeek-V3, Grok 3, and Mistral Medium 2505. Several models clustered slightly below that. The clear outlier was Cohere Command R+ with a much lower score of 4.0.

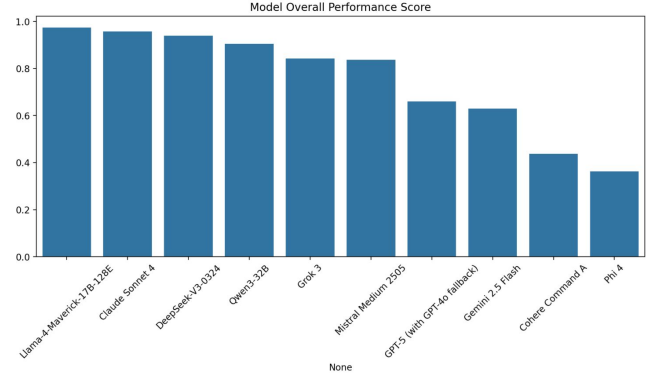
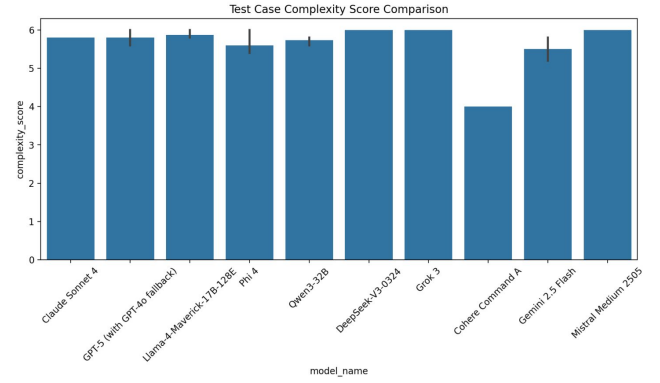
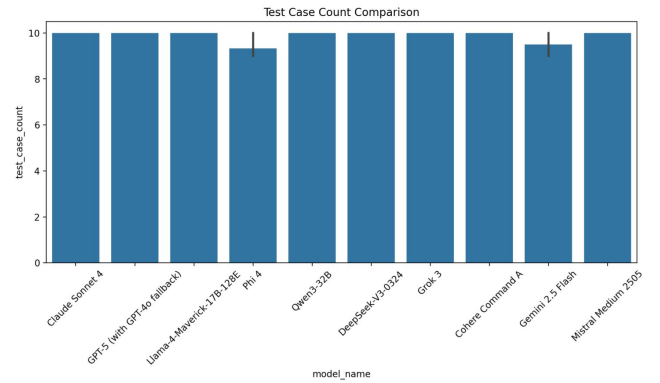
Assertions Coverage: Most models had high percentages of tests containing assertions. The most concerning was Cohere Command R+, which had 0% – none of the tests contained assertions.

Error Handling Coverage: This metric was uniformly low across the board. Even the best model (Grok 3) only included error-case tests ~27% of the time. Gemini and Cohere had 0%.

Summarizing the results: All models could generate multiple test cases, but the quality and depth of those tests varied. The best models in terms of complexity produced a wider range of scenarios and used good testing practices, but most did not explore error conditions extensively.

7 Analysis and Discussion

The experimental results provide several insights into the capabilities and limitations of modern generative AI models in test case

**Figure 2: Model Overall Performance Score****Figure 3: Testcase Complexity Score Comparison****Figure 4: Testcase Count Comparison**

generation. We discuss these insights and their implications in this section.

1. Complex Reasoning vs. Superficial Outputs: One of the goals was to evaluate the complex reasoning ability of each model – essentially, can the model not only produce code, but reason through what a thorough set of tests should include? The complexity scores and coverage metrics serve as proxies for this ability. Models like

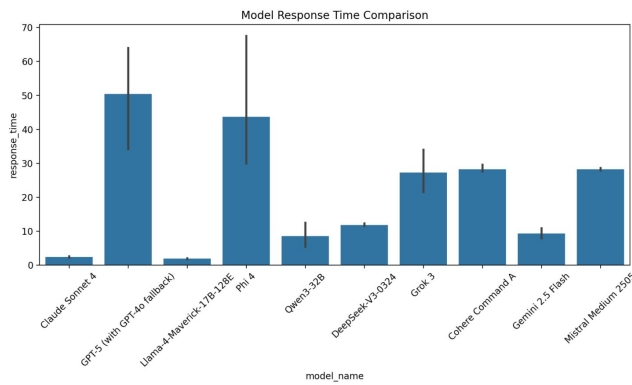


Figure 5: Model Response Time Comparison

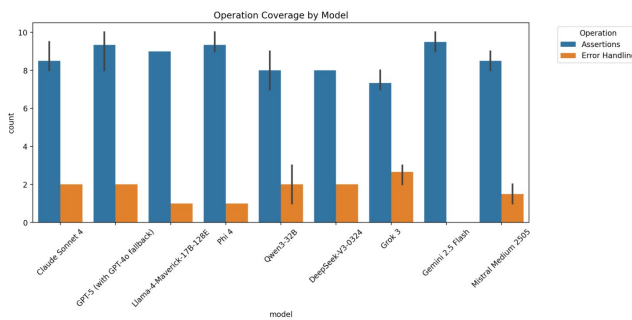


Figure 6: Operation Coverage By Model

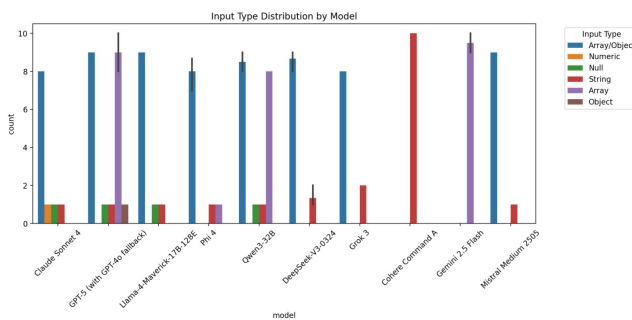


Figure 7: Input Type Distribution By Model

DeepSeek-V3, Grok 3, Mistral 2505, and LLaMA-4-Maverick scored highly, suggesting they often considered diverse test scenarios and produced well-structured tests. By manual inspection, we found that these models tended to include tests for boundary values and sometimes a special case (e.g., an empty input or a very large input). This indicates a degree of reasoning where the model is simulating possible use cases of the function in question. On the other hand, Cohere Command R+ clearly struggled – its outputs were simplistic, often just calling the function with a couple of example inputs and printing the result without any assertions. This disparity suggests that the ability to generate test cases is not merely a function of model scale, but is heavily influenced by training data composition

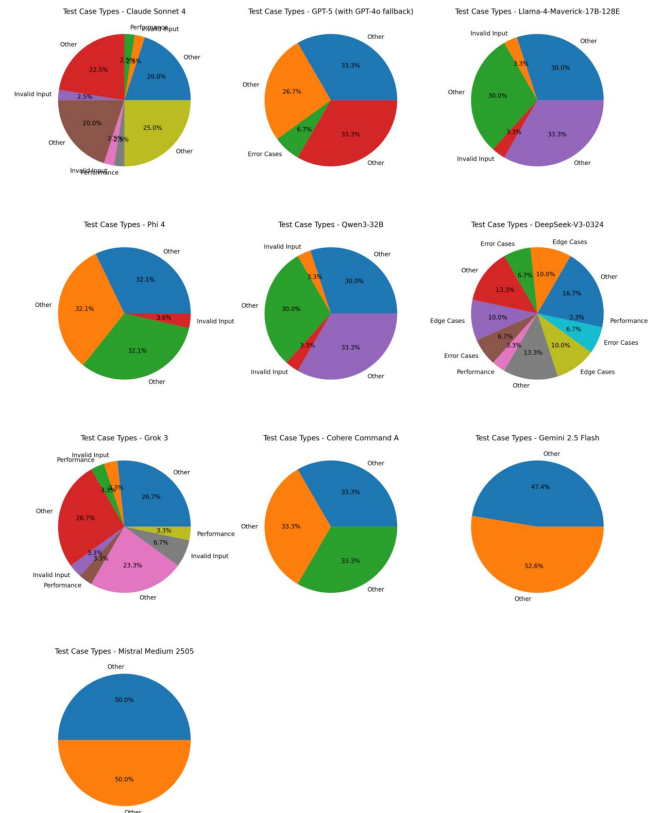


Figure 8: Pie Chart – Testcase Types

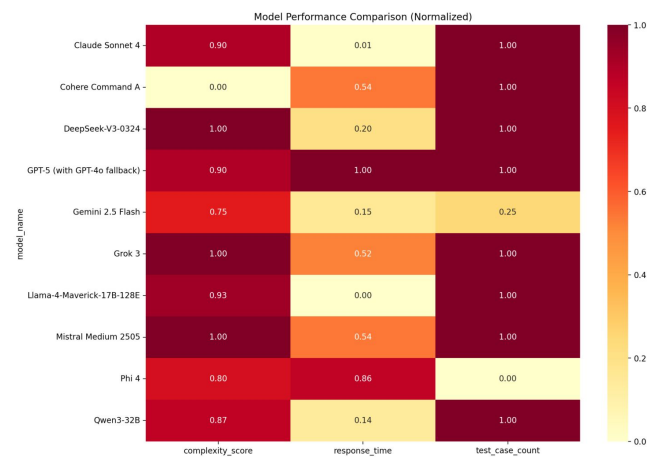


Figure 9: Model Performance Comparison (Normalized)

and instruction-tuning quality, particularly the extent to which the model has been exposed to software testing patterns during its training.

2. Importance of Assertions: The assertion coverage metric underscored a crucial point – an AI model might output something that looks like a test, but without assertions it is not a valid test.

A test that simply calls a function and prints the output does not verify correctness and will never fail, regardless of bugs in the code under test. Most models did include assertions nearly everywhere, indicating they have learned the basic pattern of a unit test. The one model that did not (Cohere R+) effectively produced useless tests, which would falsely pass every time. This finding highlights that assertion coverage should be considered a minimum requirement when evaluating AI-generated tests, and models that fail this criterion should be flagged as unreliable for testing tasks.

3. Neglect of Error Cases: Perhaps the most striking common limitation was the low frequency of error-handling tests. Even the best-performing model in this regard (Grok 3) only included error-case tests approximately 27% of the time, and several models produced none at all. This suggests that unless strongly prompted, most models focus on “happy path” testing rather than adversarial or robustness testing. In real-world software development, error handling often accounts for a substantial fraction of bugs and security vulnerabilities, so this gap represents a meaningful limitation. Practitioners relying on LLM-generated tests should be aware that manual supplementation of error-case and exception-handling tests will likely remain necessary, or alternatively, specialized prompt engineering strategies targeting negative testing scenarios could help address this deficiency.

4. Performance Trade-offs and Practicality: Response time varies dramatically across models, ranging from under 2 seconds for LLaMA-4-Maverick to over 50 seconds for GPT-5. In a real developer workflow, this difference has significant practical implications: a 2-second response enables interactive, on-demand test generation integrated into an IDE or continuous integration pipeline, whereas a 50-second response may be acceptable only for batch processing or offline test suite generation. Notably, GPT-5 did not yield clearly superior test quality compared to smaller and faster models in this setting, raising questions about the cost-effectiveness of using the largest available models for test generation tasks. The results suggest that mid-sized, well-tuned models such as LLaMA-4-Maverick, Claude Sonnet 4, and Qwen3-32B may offer the best balance of speed, quality, and cost for practical deployment.

5. Qualitative Differences: Beyond the quantitative metrics, we observed notable qualitative differences in the generated tests. Some models produced more descriptive test function names (e.g., `test_empty_list_returns_zero`) and included inline comments explaining the purpose of each test, while others followed a terse, assert-only template with minimal documentation. Additionally, certain models demonstrated an understanding of testing best practices such as test isolation and single-responsibility testing, while others occasionally combined multiple assertions testing different behaviors into a single test function. These differences can matter significantly for developer usability and long-term maintainability of generated test suites, but are not fully captured by our current quantitative metrics.

6. Impact of Model Training and Alignment: The variation in outputs reflects not just model size, but also differences in fine-tuning for instruction-following and coding tasks. Models that adhered closely to the requested structured JSON output format tended to produce higher-quality tests overall, suggesting that alignment with user instructions is correlated with testing competence. Conversely, models that frequently deviated from the requested

format or included extraneous commentary required more parser error recovery and often scored lower on complexity metrics. This observation has implications for model selection: when choosing an LLM for test generation, adherence to structured output constraints may be as important a selection criterion as raw benchmark performance on coding tasks.

7. Implications for AI-Assisted Testing Workflows: Our findings collectively suggest several practical guidelines for integrating LLMs into software testing workflows. First, teams should not rely on a single model but may benefit from combining outputs from multiple models to achieve broader coverage. Second, generated tests should always be reviewed for assertion presence and error-case coverage before integration into a test suite. Third, the choice of model should be guided by the specific use case: real-time IDE integration favors fast models, while comprehensive test suite generation for release validation may tolerate slower but more thorough models.

In conclusion, the analysis shows a landscape where multiple models can contribute to AI-assisted testing, each with different strengths. The LBST benchmarking highlights that while models can generate useful tests, none are perfect and robustness remains a common gap.

8 Limitations

While our study provides useful insights, it also has several limitations that must be acknowledged:

Benchmark Scope: We evaluated each model on three Python functions. This is a relatively small sample of possible programming tasks. A broader benchmark would strengthen conclusions.

Metric Coverage: The metrics focus on measurable aspects (counts, operations) and do not guarantee functional correctness of assertions. Executing generated tests against the source code would improve evaluation.

Complexity Score Granularity: The complexity score is heuristic and relative. Future work can refine it or replace it with code coverage and effectiveness measures.

Response Time Variability: Time can be influenced by network latency and system load; exact values are setup-dependent.

Single-Prompt Evaluation: Multi-turn refinement could improve outputs. Our one-shot setting may underestimate possible performance with iterative prompting.

No Human Preference Study: We did not collect human developer judgments, so qualitative usability is not fully captured.

Model Versions and Settings: Results are a snapshot in time, and models/settings evolve.

Cohere Model Peculiarity: Cohere Command R+ underperformed; this may reflect access/version/settings and could change with newer releases.

9 Conclusion

In this paper, we presented a comprehensive evaluation of ten modern generative AI models on the task of automated test case generation, using the Learning-Based Software Testing suite. Our study highlighted that current LLMs are capable of generating meaningful unit tests with minimal human guidance, but their reasoning depth and performance vary significantly across models.

Some models, like LLaMA-4-Maverick and Claude Sonnet 4, showed an appealing balance of speed and reasonably complex output, making them practical for near real-time use. Others, like GPT-5, incurred higher latency without a clear payoff in test quality. We also saw that domain-focused models can match or exceed flagship systems in complexity, suggesting that specialization and fine-tuning can trump generality for this task.

Crucially, our analysis revealed common shortcomings, such as the lack of error-case tests in most outputs. This points to an area of improvement both in prompting strategies and model training. The observation that assertion coverage and structured output adherence correlate with overall test quality provides actionable guidance for both model developers and practitioners seeking to integrate LLMs into their testing workflows.

Several directions for future work emerge from this study. First, we plan to broaden the benchmark to include a larger and more diverse set of target functions spanning multiple programming languages, complexity levels, and application domains (e.g., web applications, data processing pipelines, concurrent systems). Second, we intend to add executable validation of the generated tests, running them against the source code to measure actual line and branch coverage, mutation scores, and bug-detection effectiveness. Third, we will explore multi-turn and iterative prompting strategies, where model outputs are refined through follow-up instructions based on coverage gaps identified in earlier rounds. Fourth, a human evaluation study involving professional developers will be conducted to assess the readability, maintainability, and practical utility of the generated tests, complementing our automated metrics. Finally, we will keep the LBST suite updated with emerging models and techniques, providing an evolving resource for the research community to track progress in AI-assisted software testing.

Acknowledgments

The authors thank SRM Institute of Science and Technology for supporting this research.

References

- [1] OpenAI, "Introducing GPT-5," OpenAI Blog, Sep. 2025.
- [2] OpenAI, "GPT-5 System Card," Technical System Card, Sep. 2025.
- [3] OpenAI, "GPT-4o Legacy Model Reference," Developer Documentation, Aug. 2025.
- [4] Google DeepMind, "Gemini 2.5 Flash: Model Reference," Google AI Studio, 2025.
- [5] Anthropic, "Claude 4 Sonnet Model Overview," Blog Post, May 2025.
- [6] Meta AI, "LLaMA-4 Maverick (17B-128E): Technical Report," LlamaCon 2025, Apr. 2025.
- [7] Microsoft Research, "Phi-4 Technical Report," arXiv:2412.08905, 2024.
- [8] A. Yang et al., "Qwen3 Technical Report," arXiv:2505.09388, 2025.
- [9] DeepSeek AI, "DeepSeek-V3-0324 Release Notes," Product Update, Mar. 2025.
- [10] xAI, "Grok 3 Beta – The Age of Reasoning Agents," Feb. 2025.
- [11] Cohere, "Command A Model Documentation," Cohere AI Platform, 2025.
- [12] Mistral AI, "Mistral Medium 2505: Technical Overview," May 2025.
- [13] W. Wang et al., "TESTEVAL: Benchmarking Large Language Models for Test Case Generation," in *Findings of NAACL*, pp. 3547–3562, 2025.
- [14] K. Li and Y. Yuan, "Large Language Models as Test Case Generators: Performance Evaluation and Enhancement," *arXiv preprint arXiv:2404.13340*, 2024.
- [15] J. Wang et al., "Software Testing With Large Language Models: Survey, Landscape, and Vision," *IEEE Trans. Software Eng.*, vol. 50, pp. 911–936, 2024.
- [16] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed Automated Random Testing," in *Proc. ACM SIGPLAN PLDI*, pp. 213–223, 2005.
- [17] G. Fraser and A. Arcuri, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," *IEEE Trans. Software Eng.*, vol. 39, no. 2, pp. 276–291, Feb. 2013.
- [18] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Automatic Generation of High-Coverage Tests for Complex Systems Programs," in *Proc. USENIX OSDI*, pp. 209–224, 2008.
- [19] K. Claessen and J. Hughes, "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs," in *Proc. ACM ICFP*, pp. 268–279, 2000.
- [20] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-Directed Random Test Generation," in *Proc. ICSE*, pp. 75–84, 2007.
- [21] T. J. McCabe, "A Complexity Measure," *IEEE Trans. Software Eng.*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
- [22] M. H. Halstead, *Elements of Software Science*, Elsevier/North-Holland, 1977.
- [23] SonarSource SA, "Cognitive Complexity – A New Way of Measuring Understandability," White Paper, 2017.
- [24] P. Oman and J. Hagemester, "Metrics for Assessing a Software System's Maintainability," in *Proc. IEEE ICSM*, pp. 337–344, 1992.
- [25] M. Allamanis, M. Brockschmidt, and M. Khodak, "Learning to Represent Programs with Graphs," in *ICLR*, 2018.
- [26] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, 2021.
- [27] GitHub, "GitHub Copilot: AI Pair Programmer for Code and Tests," Technical Documentation, 2023.